MODULE 04 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Table types and view types

Seven kinds of table, three kinds of view, and the retention behaviour that separates them.

DOMAIN 1 of 5 WEIGHT 31% MODULE 4 of 31

admin@sthithapragna.com

Every table type is a trade between **how much protection you get** and **how much storage you pay for**.

The grid to memorise

TABLE TYPE	TIME TRAVEL (DAYS)	FAIL-SAFE	HISTORICAL DATA, MIN TO MAX
Permanent, Standard	0 or 1	7 days	7 to 8 days
Permanent, Enterprise	0 to 90	7 days	7 to 97 days
Transient	0 or 1	None	0 to 1 day
Temporary	0 or 1	None	0 to 1 day

ON THE EXAM

Fail-safe is 7 days and belongs to permanent tables only. It is never configurable and never extendable.

The details that get tested

- **The default table type is permanent.** CREATE TABLE with no keyword gives you the expensive one.
- **A transient table cannot be converted** to any other type after creation. Nor can a permanent one become transient.
- **Temporary tables are invisible outside their session**, and are dropped when it ends. The data is then unrecoverable.
- If a permanent table has **Time Travel set to 0, it enters Fail-safe immediately** when dropped.
- Once transient Time Travel passes, the data cannot be recovered **by you or by Snowflake**.

> ON THE EXAM
>
> That last point is the one candidates resist. There is no support ticket that recovers a transient table. None.

Choosing between the three

Snowflake's own guidance, which is also the exam's

- **Long-lived tables, such as fact tables, should always be permanent.** They need Fail-safe.
- **Short-lived ETL work tables can be transient**, to eliminate Fail-safe cost.
- **Session-scoped scratch data** that nothing else needs to see is temporary.
- If downtime and reload time matter, permanent may still be the better answer **despite** the Fail-safe cost.

> ON THE EXAM
>
> Shared by more than one session or job? Then it cannot be temporary, whatever the cost argument says.

The other four table types

TYPE	WHAT IT IS
External	Read-only over files in cloud storage. No DML at all. CSV, JSON, Parquet and more.
Iceberg	Apache Iceberg format in your own storage. ACID, schema evolution, readable by other engines.
Dynamic	Materialises a SELECT and keeps it fresh to a target lag. Snowflake manages the refresh.
Hybrid	Row-based, for transactional work. Enforced constraints, indexes, row-level locking. Unistore.

ON THE EXAM

External, Iceberg, dynamic and hybrid are all available in Standard Edition. None of them is an upgrade trigger.

Iceberg: the two flavours

SNOWFLAKE AS THE CATALOG

Full platform support

- Read and write, including DML
- Snowflake handles compaction and maintenance
- Best performance of the two
- The default choice for Snowflake-first work

EXTERNAL CATALOG

Limited platform support

- AWS Glue, Open Catalog, any Iceberg REST catalog
- Reached through a catalog integration
- Snowflake assumes no lifecycle management
- For lakes another system already owns

COMMON TO BOTH

Both sit on an external volume, which bundles your storage location with the IAM trust Snowflake needs.

External table or Iceberg table?

- **External table:** read-only, no DML, no transactions. A view over files in a bucket.
- **Iceberg table:** a real table format. ACID transactions, schema evolution, its own snapshots and time travel.
- Iceberg is **readable and writable by other engines**. Spark, Trino, Flink and others share the same files.
- An external-catalog Iceberg table **can be converted** to use Snowflake as the catalog later.
- A **catalog-linked database** auto-discovers and stays in sync with a remote catalog's namespaces and tables.

> ON THE EXAM
>
> If a scenario mentions other engines writing the same data, it is pointing at Iceberg, not external tables.

Tables store rows. **Views** store a query. The interesting question is what, if anything, gets materialised.

The three kinds of view

KIND	BEHAVIOUR
Standard	Stores only the definition. The query runs every time the view is referenced. No storage cost.
Materialized	Stores the results and maintains them automatically. Storage plus serverless maintenance cost. Enterprise.
Secure	Not a third thing. A property. Both standard and materialized views can be made secure.

ON THE EXAM

'Secure' is a modifier, not a category. That framing alone answers several questions.

Secure views

- They hide the **view definition itself**, not just the data. Ordinary views expose their SQL to anyone who can see the view.
- They bypass certain query optimisations, which means a documented **performance penalty**.
- **Only secure views can be shared.** A non-secure view cannot be added to a share.
- **SHOW VIEWS** has an IS_SECURE column, which is how you check.

> ON THE EXAM
>
> The optimisations a normal view enjoys can leak information about rows the user was not meant to see. That is the whole reason.

Materialized view limitations

The restriction list is the question. Learn the first line above all

- **A single table only. No joins, including self-joins.** This is the one that decides most questions.
- Cannot query another view, a materialized view, a **hybrid table**, a **dynamic table**, or a UDTF.
- No **window functions**, no UDFs, no HAVING, no ORDER BY, no LIMIT, no nested subqueries.
- **No Time Travel** on a materialized view. **Not monitored by resource monitors.** No DML, no TRUNCATE.
- Cannot be cloned directly, but **are** cloned when their schema or database is cloned.

> ON THE EXAM
>
> When a scenario needs a join and offers a materialized view, that option is wrong. It is the commonest single distractor here.

Materialized view or dynamic table?

MATERIALIZED VIEW

Always current, tightly limited

- No lag. Results are always current
- One table, no joins, no window functions
- Enterprise Edition
- Automatic, no warehouse of yours

DYNAMIC TABLE

Slight lag, very few limits

- Minimum target lag of 1 minute
- Joins, aggregations, chained pipelines
- Standard Edition
- Runs on a warehouse you name

THE TIEBREAKER

Needs a join? Dynamic table. Needs zero lag? Materialized view. That single question settles most of them.

Dynamic tables, the details

- **Minimum target lag is 1 minute**, and target lag is a target rather than a guarantee.
- **TARGET_LAG = DOWNSTREAM** refreshes an intermediate table only when a dependent needs it.
- Refresh modes: **AUTO** is the default and tries incremental, falling back to full. Also INCREMENTAL, FULL and ADAPTIVE.
- **The query definition is immutable.** Changing it means drop and recreate, and a full initial refresh.
- **Cannot be downstream from a materialized view, an external table, or a stream.**

> ON THE EXAM
>
> If the last table in a chain is set to DOWNSTREAM, nothing downstream is asking for it, so nothing ever refreshes.

Three ways to keep derived data fresh

REQUIREMENT	REACH FOR
Always-current data with no lag	Materialized view
Sub-minute freshness, or custom logic such as MERGE	Streams and tasks
Anything between: joins, aggregations, chained pipelines	Dynamic table

ON THE EXAM

This is Snowflake's own decision guide, near enough verbatim. It settles a whole family of questions.

> A nightly ETL pipeline builds a 4 TB staging table from files in a landing bucket. The table is fully rebuilt every night, and is read by three separate transformation jobs running in different sessions before being discarded.
>
> The finance team has flagged that storage costs for this table are unexpectedly high. The source files are retained in the bucket for 30 days.

Which table type should the staging table use?

A. Permanent, so that Fail-safe protects the pipeline against a failed rebuild.

B. Temporary, so that the table is purged automatically and costs nothing beyond the session.

C. Transient, with the Time Travel retention set to 0.

D. External, so that the data is never stored inside Snowflake at all.

◆ PAUSE AND DECIDE

Which table type should the staging table use?

A Permanent with Fail-safe.

Seven days of Fail-safe on 4 TB rebuilt nightly, for data reproducible from files kept for 30 days. This is the current bill.

B Temporary.

Three jobs read it in different sessions. A temporary table is invisible outside the session that created it.

C Transient, Time Travel 0.

No Fail-safe, no Time Travel storage, persists across sessions, and the data is reproducible from the retained files.

D External table.

Read-only over raw landing files. The pipeline builds a transformed table, which an external table cannot hold.

THE SIGNAL

'Different sessions' eliminated B, and '30 days of source files' made C safe. Both were planted deliberately.

> An engineer discovers that a 900 GB transient table holding three months of reconciled billing data was dropped by a faulty script four days ago. Nobody noticed until now.
>
> The table had DATA_RETENTION_TIME_IN_DAYS set to 1. The upstream system that produced the data has since been decommissioned.

What are the recovery options?

A. There are none. Transient tables have no Fail-safe, and Time Travel expired three days ago.

B. Raise a Snowflake Support ticket to recover the data from Fail-safe.

C. Use UNDROP TABLE, which works for 7 days on any table type.

D. Clone the table from a point in time four days ago using AT with an OFFSET.

◆ PAUSE AND DECIDE

What are the recovery options?

A There are none.

No Fail-safe, and Time Travel of 1 day expired three days before anyone noticed. The data is gone.

B Support can recover it from Fail-safe.

Transient tables have no Fail-safe period at all. There is nothing for Support to recover from.

C UNDROP works for 7 days on any table type.

UNDROP is bounded by Time Travel, not a fixed 7 days. Here that was 1 day, and it expired.

D Clone using AT with an OFFSET.

Cloning at a past point is itself a Time Travel operation. It is bounded by exactly the same expired window.

THE SIGNAL

The docs say it plainly: after the Time Travel period, transient data cannot be recovered by you or by Snowflake.

> A team needs a pre-computed dataset joining an orders table to a customers table, with a running total per customer. Around 40 dashboards read it, and roughly fifteen minutes of staleness is acceptable.
>
> Their first attempt was a materialized view, which failed to create. They are on Enterprise Edition.

What should they build instead, and why did the first attempt fail?

A. A secure materialized view. The original failed because materialized views must be secure to be queried by dashboards.

B. A dynamic table. Materialized views support neither joins nor window functions.

C. A standard view. The original failed because Enterprise Edition caps materialized views per schema.

D. A stream and a task. Materialized views cannot be read by more than ten concurrent sessions.

◆ PAUSE AND DECIDE

What should they build instead, and why did the first attempt fail?

A A secure materialized view.

Security is unrelated to the failure, and making it secure does not lift the join restriction.

B A dynamic table. No joins, no window functions.

Two separate violations, either alone fatal. A dynamic table has neither restriction and 15 minutes is well above the 1-minute minimum.

C A standard view, capped per schema.

No such cap exists. A standard view also recomputes on every read, which is what they were avoiding.

D Streams and tasks, concurrency limit.

No such limit. Streams and tasks would work but add custom logic the requirement never asked for.

THE SIGNAL

Fifteen minutes of acceptable staleness is the tell. It is far above the 1-minute minimum, so a dynamic table fits comfortably.

> An organisation has a large data lake in S3 in Apache Iceberg format. A Spark platform writes to it continuously, and AWS Glue is the Iceberg catalog. Several teams query the lake with Trino.
>
> They now want Snowflake to query the same data, without duplicating it and without changing which system owns the writes.

What should they create in Snowflake?

A An external table over the S3 prefix, refreshed on a schedule.

B A permanent table loaded from S3 with COPY INTO, refreshed nightly.

C An Iceberg table with Snowflake as the catalog, pointed at the same location.

D An Iceberg table using a catalog integration to AWS Glue.

◆ PAUSE AND DECIDE

What should they create in Snowflake?

A An external table over the prefix.

Ignores the Iceberg format entirely. No transactions, no schema evolution, and worse performance.

B COPY INTO a permanent table nightly.

Duplicates the data, which was explicitly ruled out, and makes Snowflake a stale copy of the lake.

C Iceberg with Snowflake as the catalog.

Would give better performance and full DML, and it takes catalog ownership away from Glue. The scenario forbids that.

D Iceberg table with a catalog integration to Glue.

Glue keeps ownership of the catalog and Spark keeps the writes. Snowflake reads the same files, no copy.

THE SIGNAL

C is the better table and the wrong answer. 'Without changing which system owns the writes' was the constraint.

What to carry into the next module

- **Fail-safe is 7 days, permanent tables only**, never configurable. Everything else maxes out at 1 day of Time Travel.
- **Transient data past its Time Travel is gone.** Not by you, not by Support. Transient cannot be converted later.
- **Secure is a property, not a view type**, and only secure views can be shared.
- **Materialized view: one table, no joins, no window functions.** That line decides most of these questions.
- **No lag, materialized view. Sub-minute or custom logic, streams and tasks. Everything else, dynamic table.**

> ON THE EXAM
>
> Next: interfaces, drivers and connectors, including the SnowSQL and Snowflake CLI pair the new exam clearly loves.